Currying

CSC345: Programming Languages & Paradigms

WCU
WEST CHESTER
UNIVERSITY

Introduction

- In Haskell, if a function takes two Int arguments and returns an Int, we write its signature as:

```
f :: Int -> Int -> Int
```

- We apply f by writing:

```
> f x y
```

- Now it's time to explain why.

Curried Form

- Functions in Haskell are represented in curried form.
- Called currying after Haskell Curry
  - Haskell Curry: one of the pioneers of the (lambda) λ-calculus.

- **Big Idea:** functions take their arguments *one at a time*.
- That is, **every function in Haskell takes *exactly* one argument**.
- (Called the “curried representation of functions”)

Example

```
add :: Int -> Int -> Int
add x y = x + y
```

```
> add 3 4
3 + 4
7
```

Prompt: WAIT! there seems to be two arguments?

"Under the hood"

add :: Int -> Int -> Int

is shorthand for

add :: Int -> (Int -> Int)

(The -> arrow associates to the right.)

```
add :: Int -> (Int -> Int)
```

- Q: what does this type signature mean?
- A: add takes one argument, and returns a function
  - add is higher-order

- Function definition remains the same!

```
add x y = x + y
```

- Calling the function:

```
> add 3 4
(add 3) 4
```

- Because add only takes one argument…
- Returns a function of type (Int -> Int)
- This functions is the “3+” function, which takes one argument, and returns that value with 3 added to it

```
add :: Int -> (Int -> Int)
```

```
add 3 4
```

```
Int
```

Function Application is Left Associative

add 3 4 is the same as (add 3) 4

- add 3 makes sense by itself!
  - It is the function that ”adds 3 to values”
- We just showed how 2-argument functions can be defined in terms of 1-argument functions.

General Form

Left Associative $f\ e_1\ e_2\ ..\ e_n$ $(\ ..\ ((f\ e_1)\ e_2)\ ..\ e_n)$

shorthand for

Right Associative $t_1 \rightarrow t_2 \rightarrow .. \rightarrow t_n$ $t_1 \rightarrow (t_2 \rightarrow (\ ..\ (t_n \rightarrow t)\ ..\ ))$

a -> b -> c means a -> (b -> c) not (a -> b) -> c

Another way to look at currying:

```
add :: Int -> Int -> Int
add x y = x + y
```

```
> add 3 4
```

… means same as:

```
add :: Int -> (Int -> Int)
add x = g
  where
    g y = x + y
```

```
> (add 3) 4
```

Curried Form

- 1 argument function
- Return a function if there are "multiple arguments"

Uncurried Form

- Bundle multiple arguments into one via a pair/tuple

```
add :: Int -> Int -> Int
add x y = x + y
```

```
add :: (Int, Int) -> Int
add (x,y) = x + y
```

Usually prefer curried form – and everything that "we get for free"

1. Notation is neater. No extra parentheses necessary.
2. Permits <u>partial application</u>

```
> :type (add 3) Int ->
Int
```

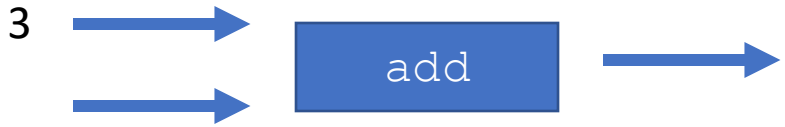
```
> :type (add 3 4)
Int
```

Another Perspective

Can view the `add` function as a box with 2 arguments and 1 result (ignoring the currying)

Can apply the `add` function to 2 arguments (automatic currying)

Or apply the `add` function to 1 argument (partial application)



Example

add 3
Has type: (Int -> Int)
or
Int -> Int

2 partial applications:

map (add 3)
Has type: [Int] -> [Int]

A full application:

```
addThree :: [Int] -> [Int]
addThree = map (add 3)

> addThree [1,2,3]
[4,5,6]
```

Example 2: (using `foldr`!)

```
foldr :: (a -> a -> a) -> a -> [a] -> a
foldr f v [] = v
foldr f v (x:xs) = f x (foldr f v xs)
```

```
sum :: [Int] -> [Int]
sum xs = foldr (+) 0 xs
```

… now using partial application …

```
sum :: [Int] -> [Int]
sum = foldr (+) 0
```

Will return a function that takes a list of Ints and produces an Int, that is [Int] -> Int.

Comparison

```
sum :: [Int] -> [Int]
sum xs = foldr (+) 0 xs
```

```
> sum [1,2,3]
                    ..
foldr (+) 0 [1,2,3]
```

```
sum :: [Int] -> [Int]
sum = foldr (+) 0
```

```
> sum [1,2,3]
                    ..
(foldr (+) 0) [1,2,3]
```